

第 1 周：需求理解、技术设计与项目启动

本周目标

完成从需求理解到技术设计的首次闭环，并初始化本地开发环境和项目协作流程。

本周详细需求

设计文档要写什么

第一周不急着写业务代码，先把设计想清楚。以下文档都要产出，而且互相对得上——不能 API 设计里写了一个字段，数据库表里找不到。

1. **需求疑问清单**：读 PRD 时有任何不清楚的地方都记下来，标"待确认"或"已确认"，确认了的写清楚结论是怎么来的。
2. **业务名词表**：把项目里反复出现的词（任务、同步、幂等、Worker 等）统一定义，团队里大家说的是同一个东西。
3. **系统边界图**：画清楚 React Web、API、Worker、MySQL、Redis 各自管什么，哪些是 HTTP 同步调用，哪些是 Redis 异步消息。
4. **核心流程图**：从用户上传 CSV 到任务结束的完整链路，正常路径走通，主要异常分支也要画出来。
5. **API 设计**：把计划做的接口都列出来，写清请求方法、路径、参数、响应体、状态码和错误码。用 OpenAPI 规范写。
6. **数据库设计**：每张表的字段、类型、约束、索引都列出来，索引要写为什么加。
7. **任务状态机**：有哪些状态、什么条件下从一个状态转到另一个状态。
8. **Web 页面设计**：有哪些页面、路由是什么、每个页面的核心交互、加载/空数据/出错时分别显示什么。
9. **测试计划**：正常场景、异常场景、边界场景分别怎么测，哪些是单元测试、哪些是集成测试、哪些是前端交互测试。

API 设计的几条规矩

- 接口走 RESTful 风格，路径统一以 `/api/v1/` 开头。
- 成功响应包一层 `{ "data": ..., "meta": ... }`，失败响应包一层 `{ "error": { "code", "message", "details" } }`。前后端都按这个格式来，别各写各的。
- 错误码用大写下划线的字符串（比如 `JOB_NOT_FOUND`），别用数字错误码——数字码过两周没人记得什么意思。
- 列表接口都要支持分页，参数就叫 `page` 和 `page_size`。

- 接口返回里不要出现服务器文件路径、SQL 语句、异常堆栈或数据库连接信息，这些是内部细节。
- 创建任务的接口要快速返回，别等整个 CSV 处理完才响应。
- API 文档用 OpenAPI 3.x 写，能生成可交互的文档页面最好。

数据库设计的几条规矩

- 任务、成功记录、错误记录、执行尝试分开建表，别塞一张表里，查起来方便也不容易锁冲突。
- 金额用 `DECIMAL`，别用 `FLOAT` 或 `DOUBLE`，浮点数存金额会丢精度。
- 时间字段统一用 `datetime(3)` 存 UTC，前端展示时再转本地时区。
- 任务表主键用 UUID 或 ULID（字符串），业务数据表用自增主键就行。
- 每张表都加上 `created_at` 和 `updated_at`，排查问题时全靠它们。
- 表之间的关联在应用层维护，不强制建数据库外键，但该加的索引要加。
- 该加唯一约束的地方加（比如同一任务内的业务标识），这是防并发重复写入的最后一道防线。
- 索引别瞎加，每个索引都要能说清是给哪个查询用的——索引多了写入会慢。
- 数据库迁移脚本要能重复执行（幂等），用迁移工具或者版本化脚本管起来，别手写 SQL 往库里怼。

项目结构和工程规范

- 后端、前端、Docker 配置分开目录放，别混在一起。
- 配置从环境变量读，提供 `.env.example` 列全所有需要的配置项，`.env` 本身不要提交到仓库。
- Docker Compose 能一键拉起 Web、API、Worker、MySQL、Redis 五个服务。
- README 写清楚：项目是干嘛的、技术栈、环境要求、怎么启动、怎么停、怎么做数据库迁移、怎么构建前端、怎么跑测试、怎么看日志、怎么清数据。
- 一个新人照着 README 应该能在 15 分钟内把项目跑起来。如果做不到，说明 README 写得不够细。
- 提交代码前先跑格式化和静态检查，别把带 warning 的代码往上推。
- 分支命名：`feature/xxx`、`fix/xxx`、`chore/xxx`，见名知意。
- 每个 PR 关联对应的 Issue，描述里写清改了什么、怎么测的、有 UI 变化就截图。

配置项清单

`.env.example` 里要包含以下所有配置项，后续会逐个用到。第一周可以先给默认值，实现到对应功能时再接入：

配置项	默认值	说明
<code>MYSQL_HOST</code>	<code>mysql</code>	MySQL 主机
<code>MYSQL_PORT</code>	<code>3306</code>	MySQL 端口

配置项	默认值	说明
MYSQL_DATABASE	syncflow	数据库名
MYSQL_USER	syncflow	数据库用户
MYSQL_PASSWORD	syncflow123	数据库密码（仅本地开发用）
REDIS_ADDR	redis:6379	Redis 地址
MAX_UPLOAD_FILE_SIZE_MB	10	上传文件大小上限（MB）
MAX_RECORDS_PER_JOB	10000	单任务最大有效数据行
WORKER_CONCURRENCY	2	Worker 并发数
JOB_TIMEOUT_SECONDS	300	单任务执行超时（秒）
WORKER_SHUTDOWN_TIMEOUT_SECONDS	30	Worker 优雅停止超时（秒）
MAX_JOB_ATTEMPTS	3	任务最大执行次数（含首次）
RETRY_BACKOFF_SECONDS	5,30,120	重试退避时间（秒，逗号分隔）
IDEMPOTENCY_KEY_TTL_HOURS	24	幂等键有效期（小时）

`.env` 里的值可以跟 `.env.example` 不一样，但 `.env` 绝对不能提交到仓库。

设计阶段就要考虑的非功能需求

这些不是等功能做完了再补的，设计的时候就得想进去。后续会逐步实现，但第一周的设计文档里要体现。

本地能跑起来 – `docker compose up` 能把五个服务都拉起来。 – 所有依赖本地就能跑，不要把核心功能架在云服务、外部数据库或者个人账号上。

前端代码质量 – 用 TypeScript，页面、组件、API 客户端、类型定义分层放清楚。 – API 的响应类型集中定义，别每个页面自己写一套。 – 任务状态展示、日期格式化、错误提示这些通用逻辑抽出来复用，别在好几个页面复制粘贴。 – 生产构建要能过，TypeScript 类型错误零容忍。

安全 – 仓库里不能出现真实密码、密钥、数据库数据或者 `.env` 文件。 – 上传的文件存在受控目录里，文件名要做规范化，防目录穿越（`../../etc/passwd` 这种）。 – API 不返回内部路径、SQL、堆栈、连接配置。 – 日志里不打密码、完整数据库 DSN 或敏感环境变量。

可观测性 – 日志至少带时间、级别、服务名、消息内容。 – 跟任务相关的日志一定要带上 `job_id`，不然出了问题根本没法查。 – 每次任务执行尝试都能从日志和数据库记录里串起来。 – 有 `/healthz` 做进程存活检查，有余力再加 `/readyz` 检查 MySQL 和 Redis 通不通。

性能底线 – 处理 1,000 行合法 CSV 服务不能崩。 – 处理 10,000 行 CSV 不能一口气全读进内存，用流式读取或者分批处理。 – 列表和错误明细接口必须分页。

怎么协作

阅读 PRD -> 提需求问题 -> 技术设计 -> 数据库与 API 设计
-> 分支开发 -> 本地测试 -> 提 PR -> AI Code Review
-> 合并发布 -> 复盘

- 所有改动走分支 -> PR -> AI Code Review 流程，别直接往主分支推。
- PR 先过 AI Code Review，再由人做最终审核；AI 提的问题要逐条看，有道理就改，没道理要说明为什么不改。
- Review 重点看：业务逻辑对不对、错误处理全不全、测试有没有覆盖、代码读不读得懂、跟设计文档一不一致。

任务

1. 阅读 docs/01-product-requirements.md 。
2. 整理业务名词、用户角色和核心流程。
3. 完成 Go 或 Python 开发环境，以及 Node.js、React 和 TypeScript 开发环境。
4. 通过 Docker Compose 启动 MySQL 8 和 Redis 7，并创建本地开发数据库。
5. 创建后端与 React 前端项目骨架、基础 README 和统一启动脚本。
6. 实现 /healthz 接口。
7. 创建第一个分支并提交第一个 PR。
8. 绘制系统边界图和核心流程图。
9. 编写 API 设计，包括请求、响应、状态码和错误码。
10. 编写 MySQL 数据库设计和任务状态机。
11. 编写异常场景与测试计划。

产出物

- 需求与技术设计包：需求疑问、名词表、技术设计、API、数据库、Web 页面、状态机和测试计划。
- 可运行项目骨架：React 与后端工程、Docker Compose、MySQL、Redis、环境配置、健康检查和 README。
- ** 公开代码仓库：所有产出物均提交在此仓库，以代码+文档为主。 **

产出物细则

产出物	必须包含	完成判定
需求与技术设计包	疑问清单、名词表、模块边界、API、数据库、状态机、页面设计、测试计划	各文档相互一致，且每条核心需求可追溯到接口、数据结构和测试场景
可运行项目骨架	React、后端、Web/API/Worker/MySQL/Redis Compose 配置、 <code>.env.example</code> 、 <code>/healthz</code> 、README	<code>docker compose up -d</code> 后依赖健康，Web/API 可访问，健康检查返回 200

验收标准

- MySQL 和 Redis 可以在本地启动。
- Web、API 和 Worker 均可通过统一命令启动；API 返回健康状态。
- 新成员能够按照 README 完成启动。
- 能够口头说明项目目标、用户、核心流程和当前版本范围。
- 需求疑问有明确的待确认或已确认状态。
- 技术设计能够说明 Redis 队列与 Worker 的异步流程、MySQL 事务边界和 API 错误处理。
- 任务状态机、数据库设计、Web 页面交互和测试场景彼此一致。

AI 使用要求

可以用 AI 帮你解释技术概念、生成环境配置草稿、排查报错，但生成的东西要自己跑一遍验证——端口对不对、数据库连不连得上、启动命令能不能跑通，都要确认。AI 生成的项目模板如果自己讲不清每一行是干嘛的，就别提交。